

# Deep Learning

## Tokenização em Modelos de Linguagem

Lucas Silveira Kupssinskü

Machine Learning Theory and Applications Laboratory  
PUCRS

abril de 2026

# Visão Geral

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

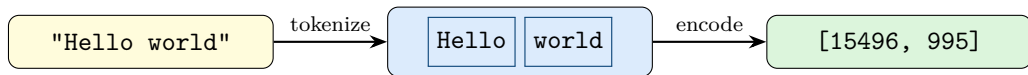
# O que é Tokenization?

---

Tokenization é o processo de converter texto bruto em uma sequência de unidades discretas (**tokens**) que um modelo pode processar.

O tokenizer define:

1. O **vocabulário** — quais tokens existem
2. A **segmentação** — como o texto é dividido em tokens
3. O **mapeamento** — tokens  $\leftrightarrow$  IDs inteiros



# Por que isso importa?

---

A tokenization **molda silenciosamente** tudo que vem a seguir:

## Problemas causados pela tokenization

- O GPT não consegue contar letras de forma confiável em "strawberry" (3 r's)
- Desigualdade multilíngue: o inglês usa menos tokens que outros idiomas
- "123 + 456" pode ser dividido de forma inconsistente entre os dígitos

## Trade-offs de design

- Vocabulário pequeno → sequências longas
- Vocabulário grande → embeddings esparsos
- Dificilmente encontraremos uma “granularidade correta”

**O tokenizer não é um detalhe de pré-processamento — é uma decisão arquitetural.**

# O Problema da Fertility

**Fertility** = número de tokens necessários para representar uma palavra ou frase.

| Idioma   | Frase                 | Tokens GPT-2 |
|----------|-----------------------|--------------|
| Inglês   | “Hello, how are you?” | 6 tokens     |
| Espanhol | “Hola, ¿cómo estás?”  | 9 tokens     |
| Japonês  | (equivalente)         | 14+ tokens   |

## Consequência

Maior fertility = mais tokens = mais custo computacional, janela de contexto efetiva menor, e frequentemente pior desempenho para esse idioma.

# Overview

---

1. Por que a Tokenization Importa
- 2. Abordagens Ingênuas**
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

# Nosso Exemplo Recorrente

---

Ao longo desta aula usamos o mesmo **corpus de treinamento**:

| Palavra | Frequência no corpus |
|---------|----------------------|
| low     | 5                    |
| lower   | 2                    |
| newest  | 6                    |
| widest  | 3                    |

## Por que as frequências importam

Um tokenizer é **aprendido** a partir de um corpus. A frequência de cada palavra (e de cada par de caracteres) determina quais tokens são adicionados ao vocabulário.



# Tokenization em Nível de Caractere

**Ideia:** Cada caractere é um token.

## Exemplo Recorrente

"low lower newest widest"

→ 

|   |   |   |   |
|---|---|---|---|
| l | o | w | _ |
|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| l | o | w | e | r | _ |
|---|---|---|---|---|---|

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| n | e | w | e | s | t | _ |
|---|---|---|---|---|---|---|

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| w | i | d | e | s | t |
|---|---|---|---|---|---|

Vocabulário: { l, o, w, \_, e, r, n, s, t, i, d }     $|\mathcal{V}| = 11$

### ✓ Vantagens

- Vocabulário reduzido ( $\sim 256$  para ASCII)
- Sem palavras fora do vocabulário (OOV)
- Agnóstico ao idioma

### × Desvantagens

- Sequências muito longas
- Perde a semântica em nível de palavra
- O modelo precisa “aprender” a ortografia

# Tokenization em Nível de Palavra

**Ideia:** Dividir por espaço em branco (e pontuação). Cada palavra é um token.

## Exemplo Recorrente

"low lower newest widest"

→ 

|     |       |        |        |
|-----|-------|--------|--------|
| low | lower | newest | widest |
|-----|-------|--------|--------|

Vocabulário: { low, lower, newest, widest }     $|\mathcal{V}| = 4$

### ✓ Vantagens

- Sequências curtas
- Tokens carregam significado semântico
- Simples de implementar

### × Desvantagens

- Vocabulário gigantesco (100K+)
- Não lida com palavras OOV
- Sem compartilhamento morfológico: low e lowest são tokens sem relação

**A sacada principal:** Queremos algo *entre* caracteres e palavras.

⇒ **Tokenization por subwords**

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
- 3. Byte Pair Encoding (BPE)**
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

# BPE — Ideia Central

**Byte Pair Encoding** (Sennrich et al., 2016) mescla iterativamente o par de símbolos adjacentes mais frequente.

## Algoritmo

1. Começa com um vocabulário de todos os caracteres individuais
2. Conta todos os pares de símbolos adjacentes no corpus
3. Mescla o par mais frequente em um novo símbolo
4. Repete os passos 2–3 por  $k$  merges (hiperparâmetro)

## Propriedades principais:

- Bottom-up, guloso, baseado em frequência
- A tabela de merges é aprendida a partir de um corpus de treinamento
- Codificação determinística na inferência (aplica os merges na ordem aprendida)

**Utilizado em:** GPT, GPT-2, GPT-3, RoBERTa, CodeGen, ...

## BPE — Configuração do Exemplo Simplificado

**Corpus de treinamento** (com frequências de palavras e marcador de fim de palavra `_`):

| Palavra | Freq | Divisão inicial |   |   |   |   |   |   |
|---------|------|-----------------|---|---|---|---|---|---|
| low_    | 5    | l               | o | w | _ |   |   |   |
| lower_  | 2    | l               | o | w | e | r | _ |   |
| newest_ | 6    | n               | e | w | e | s | t | _ |
| widest_ | 3    | w               | i | d | e | s | t | _ |

**Vocabulário inicial:**  $\mathcal{V} = \{ l, o, w, _, e, r, n, s, t, i, d \}$   $|\mathcal{V}| = 11$

Vamos executar os merges passo a passo...

# BPE — Merge Passo 1

Contagem de todos os pares adjacentes:

| Par    | Contagem | Par    | Contagem | Par    | Contagem | Par    | Contagem |
|--------|----------|--------|----------|--------|----------|--------|----------|
| (e, s) | 9        | (l, o) | 7        | (o, w) | 7        | (s, t) | 9        |
| (e, w) | 6        | (w, _) | 5        | (t, _) | 9        | (e, r) | 2        |
| (w, e) | 8        | (n, e) | 6        | (w, i) | 3        | (d, e) | 3        |
| (i, d) | 3        | (r, _) | 2        |        |          |        |          |

Mais frequentes: (e, s), (s, t) e (t, \_) têm contagem 9.

(Desempate: escolher (e, s) primeiro.)

Merge 1: (e, s) → **es** (contagem: 9)

|         |   |   |   |   |           |   |   |  |
|---------|---|---|---|---|-----------|---|---|--|
| low_    | 5 | l | o | w | _         |   |   |  |
| lower_  | 2 | l | o | w | e         | r | _ |  |
| newest_ | 6 | n | e | w | <b>es</b> | t | _ |  |
| widest_ | 3 | w | i | d | <b>es</b> | t | _ |  |

$\mathcal{V} = \{ l, o, w, _, e, r, n, s, t, i, d, es \} \quad |\mathcal{V}| = 12$

## BPE — Merges Passos 2–4

Merge 2: (es, t) → **est** (contagem: 9)

|         |   |   |   |   |            |   |
|---------|---|---|---|---|------------|---|
| newest_ | 6 | n | e | w | <b>est</b> | - |
| widest_ | 3 | w | i | d | <b>est</b> | - |

(demais inalterados)

Merge 3: (est, \_) → **est\_** (contagem: 9)

|         |   |   |   |   |             |
|---------|---|---|---|---|-------------|
| newest_ | 6 | n | e | w | <b>est_</b> |
| widest_ | 3 | w | i | d | <b>est_</b> |

Merge 4: (l, o) → **lo** (contagem: 7)

|        |   |           |   |   |   |   |
|--------|---|-----------|---|---|---|---|
| low_   | 5 | <b>lo</b> | w | - |   |   |
| lower_ | 2 | <b>lo</b> | w | e | r | - |

$\mathcal{V} = \{ l, o, w, _, e, r, n, s, t, i, d, es, est, est_, lo \}$   $|\mathcal{V}| = 15$

## BPE — Merges Passos 5–7

Merge 5: (lo, w) → **low** (contagem: 7)

|        |   |            |   |   |   |
|--------|---|------------|---|---|---|
| low_   | 5 | <b>low</b> | - |   |   |
| lower_ | 2 | <b>low</b> | e | r | - |

Merge 6: (n, e) → **ne** (contagem: 6)

|         |   |           |   |      |  |
|---------|---|-----------|---|------|--|
| newest_ | 6 | <b>ne</b> | w | est_ |  |
|---------|---|-----------|---|------|--|

Merge 7: (ne, w) → **new** (contagem: 6)

|         |   |            |      |  |
|---------|---|------------|------|--|
| newest_ | 6 | <b>new</b> | est_ |  |
|---------|---|------------|------|--|

Após 7 merges, o corpus é representado como:

|            |   |     |            |   |   |   |     |            |      |     |   |   |   |      |     |
|------------|---|-----|------------|---|---|---|-----|------------|------|-----|---|---|---|------|-----|
| <b>low</b> | - | (5) | <b>low</b> | e | r | - | (2) | <b>new</b> | est_ | (6) | w | i | d | est_ | (3) |
|------------|---|-----|------------|---|---|---|-----|------------|------|-----|---|---|---|------|-----|

Observe como **est\_** é compartilhado entre “newest” e “widest”!



# BPE — Codificando Novo Texto

Na inferência, aplica-se os merges aprendidos **em ordem**:

## Codificando a palavra “lowest”

1. Início: 

|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| l | o | w | e | s | t | _ |
|---|---|---|---|---|---|---|
2. Aplicar merge 1 (e,s)→es: 

|   |   |   |    |   |   |
|---|---|---|----|---|---|
| l | o | w | es | t | _ |
|---|---|---|----|---|---|
3. Aplicar merge 2 (es,t)→est: 

|   |   |   |     |   |
|---|---|---|-----|---|
| l | o | w | est | _ |
|---|---|---|-----|---|
4. Aplicar merge 3 (est,\_)→est\_: 

|   |   |   |      |
|---|---|---|------|
| l | o | w | est_ |
|---|---|---|------|
5. Aplicar merge 4 (l,o)→lo: 

|    |   |      |
|----|---|------|
| lo | w | est_ |
|----|---|------|
6. Aplicar merge 5 (lo,w)→low: 

|     |      |
|-----|------|
| low | est_ |
|-----|------|

**Resultado:**

|     |      |
|-----|------|
| low | est_ |
|-----|------|

 — compartilha subwords com “low”, “newest”, “widest”!

Mesmo que “lowest” **nunca tenha aparecido no corpus de treinamento**, o modelo consegue compô-la a partir de subwords aprendidos.

# BPE — Esboço em Python

---

```
import re, collections

def get_stats(vocab):
    """Conta a frequência de pares de símbolos adjacentes."""
    pairs = collections.Counter()
    for word, freq in vocab.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i+1])] += freq
    return pairs

def merge_vocab(pair, vocab):
    """Mescla todas as ocorrências de um par no vocabulário."""
    bigram = re.escape(' '.join(pair))
    pattern = re.compile(r'(?!\S)' + bigram + r'(?!\S)')
    new_vocab = {}
    for word in vocab:
        new_word = pattern.sub(' '.join(pair), word)
        new_vocab[new_word] = vocab[word]
    return new_vocab

# Corpus de treinamento (frequências de palavras)
vocab = {'l o w _': 5, 'l o w e r _': 2,
         'n e w e s t _': 6, 'w i d e s t _': 3}

num_merges = 7
for i in range(num_merges):
    pairs = get_stats(vocab)
    best = max(pairs, key=pairs.get)
    vocab = merge_vocab(best, vocab)
    print(f"Merge {i+1}: {best} -> {' '.join(best)}")
```

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
- 4. WordPiece**
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

## WordPiece — Ideia Central

**WordPiece** (Schuster & Nakajima, 2012; Wu et al., 2016) é similar ao BPE, mas seleciona merges com base na **likelihood** em vez da frequência.

### Diferença principal em relação ao BPE

Em vez de mesclar o par mais *frequente*, o WordPiece mescla o par que **maximiza a likelihood dos dados de treinamento**:

$$\text{score}(x, y) = \frac{\text{freq}(xy)}{\text{freq}(x) \cdot \text{freq}(y)}$$

Isso favorece mesclar pares cuja combinação é muito mais provável do que o acaso.

### Outras diferenças:

- Usa o prefixo **##** para tokens de continuação: “playing” → play ##ing
- Longest-match guloso da esquerda para a direita na inferência

**Utilizado em:** BERT, DistilBERT, Electra, ...

## WordPiece — Exemplo Simplificado

Mesmo corpus, scores diferentes:

| Par    | freq(xy) | freq(x)·freq(y)     | Score                  |
|--------|----------|---------------------|------------------------|
| (e, s) | 9        | $16 \times 9 = 144$ | 0.0625                 |
| (s, t) | 9        | $9 \times 9 = 81$   | 0.111                  |
| (i, d) | 3        | $3 \times 3 = 9$    | <b>0.333</b> ← highest |
| (n, e) | 6        | $6 \times 16 = 96$  | 0.0625                 |
| (l, o) | 7        | $7 \times 7 = 49$   | 0.143                  |

O WordPiece mescla (i, d) primeiro, não (e, s)!

### Intuição

(i, d) aparecem quase *exclusivamente* juntos → alta informação mútua.

(e, s) aparecem com frequência, mas também de forma independente → score menor.

## WordPiece — Inferência: Longest Match

Na inferência, o WordPiece usa **longest-match-first guloso** da esquerda para a direita.

Exemplo: codificando “unhappiness” com um vocabulário estilo BERT

Suponha que o vocabulário contém: un, happy, ##happi, ##ness, ##ly, ...

1. Varredura da esquerda: maior correspondência = un

2. Restante: “happiness”. Maior correspondência = ##happi

3. Restante: “ness”. Maior correspondência = ##ness

Resultado: un ##happi ##ness

O prefixo ## indica “este token é uma continuação da palavra anterior” (não é o início de uma palavra).

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
- 5. Unigram Language Model (SentencePiece)**
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

# Unigram — Ideia Central

**Unigram LM** (Kudo, 2018) adota a **abordagem oposta** à do BPE:

## Algoritmo

1. Começa com um vocabulário inicial **grande** (ex.: todos os substrings até certo comprimento)
2. Define um modelo de linguagem unigrama:  $P(x) = \prod_{i=1}^n P(x_i)$
3. Para cada token, calcula o quanto a likelihood **cai** se ele for removido
4. Remove os tokens com a **menor perda** (menos úteis)
5. Repete até o vocabulário atingir o tamanho desejado

## Diferença principal em relação ao BPE

BPE: começa pequeno, cresce (bottom-up)

Unigram: começa grande, poda (top-down)



## Unigram — Exemplo Simplificado (1/2)

---

**Corpus:** “low” (freq 10), “er” (freq 10), “lower” (freq 1)

**Passo 1:** Começa com vocabulário de todos os substrings;  $P(t) = \frac{\text{count}(t)}{N}$ , onde  $N = \sum_{t'} \text{count}(t')$ :

| Token | $P(\cdot)$ | Token | $P(\cdot)$ | Token | $P(\cdot)$ |
|-------|------------|-------|------------|-------|------------|
| low   | 0.105      | lo    | 0.105      | l     | 0.105      |
| er    | 0.105      | ow    | 0.105      | o     | 0.105      |
| lower | 0.010      | we    | 0.010      | w     | 0.105      |
| owe   | 0.010      | wer   | 0.010      | e     | 0.105      |
| lowe  | 0.010      | ower  | 0.010      | r     | 0.105      |

Substrings de “low” ou de “er” têm count = 11 (= 10 ocorrências da sua palavra +1 de “lower”); substrings exclusivas de “lower” têm count = 1;  $N = 9 \times 11 + 6 \times 1 = 105$ .

## Unigram — Exemplo Simplificado (2/2)

---

**Corpus:** “low” (freq 10), “er” (freq 10), “lower” (freq 1)

**Passo 2:** Melhor segmentação de “lower” sob este modelo:

$$P(\text{low}) \times P(\text{er}) = 0.105 \times 0.105 \approx \mathbf{0.011} \quad \leftarrow \text{melhor}$$

$$P(\text{lower}) = 0.010 \quad (\text{token único; ligeiramente inferior ao par})$$

$$P(\text{l}) \times P(\text{owe}) \times P(\text{r}) = 0.105 \times 0.010 \times 0.105 \approx 0.0001$$

**Passo 3:** Poda do vocabulário:

$$P(\text{owe}) = 0.010 \text{ — nunca entra na melhor segmentação} \rightarrow \text{podá-lo.}$$

$$P(\text{low}) = 0.105 \text{ — essencial em 11 segmentações} \rightarrow \text{mantê-lo.}$$

# SentencePiece — O Framework

**SentencePiece** (Kudo & Richardson, 2018) é um **framework**, não um algoritmo.

## O que torna o SentencePiece especial?

1. **Agnóstico ao idioma:** trata a entrada como um fluxo bruto de bytes/unicode
2. **Sem pre-tokenization:** não depende de divisão por espaço em branco
3. O espaço em branco é tratado como um caractere regular (substituído por \_)
4. Suporta **tanto** BPE quanto Unigram como algoritmo subjacente

## Exemplo

"New York" → \_New \_York

O \_ codifica o espaço, tornando a tokenization **totalmente reversível**.

**Utilizado em:** T5, LLaMA, ALBERT, XLNet, Gemma, ...

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
- 6. Byte-Level BPE**
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
9. Resumo

# Byte-Level BPE — Motivação

**Problema:** O BPE padrão opera sobre caracteres Unicode. Mas o Unicode tem 150K+ pontos de código — o vocabulário base é enorme, e scripts raros geram muitos tokens <unk>.

**Solução:** Byte-Level BPE (Radford et al., 2019 — GPT-2)

- Opera sobre **bytes brutos** (sempre 256 tokens base)
- Todo texto pode ser codificado — **nunca há tokens desconhecidos**
- Em seguida, aplica-se os merges padrão do BPE sobre os bytes

## Exemplo

O emoji “:)” em UTF-8 é 0x3A 0x29:

→ 0x3A 0x29 (2 tokens de byte)

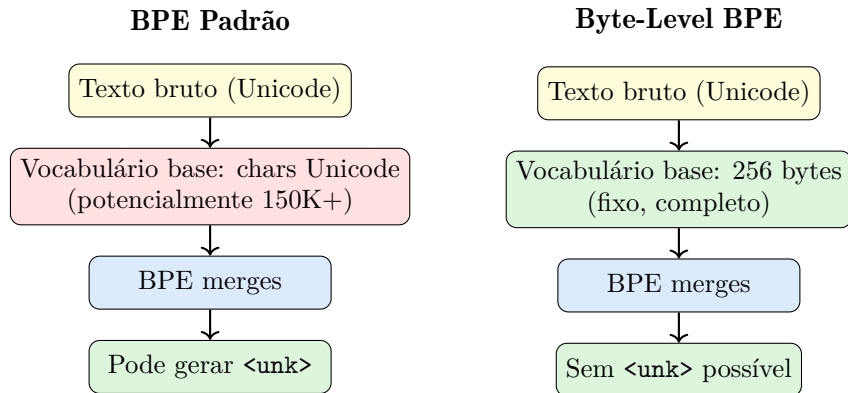
Um caractere chinês como “[CJK]” pode ter 3 bytes UTF-8:

→ 3 tokens de byte (ou mesclados se forem suficientemente frequentes)

**Utilizado em:** GPT-2, GPT-3, GPT-4, Codex, LLaMA 2/3, ...

# Byte-Level BPE vs. BPE Padrão

---



O GPT-2 usa um **mapeamento byte-para-unicode**: cada byte é representado como um caractere Unicode imprimível, de modo que o código BPE baseado em strings existente funciona sem modificações.

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
- 7. Comparação e Considerações Práticas**
8. De Tokens a Embeddings
9. Resumo

# OOV e Pré-tokenização

## Tratamento de OOV (Out-Of-Vocabulary)

Tokens ou palavras que **não apareceram** no corpus de treinamento.

- **Caractere:** cobertura total — o vocabulário são os próprios bytes/chars, nada é OOV
- **BPE / Unigram:** decompõe até chegar nos caracteres do vocabulário base (*chars raros*)
- **WordPiece:** substitui pelo token especial [UNK] se não conseguir segmentar

## Pré-tokenização

Etapa **anterior** ao algoritmo: divide o texto em “palavras” por espaço e/ou pontuação.

- **WordPiece (Sim):** o BERT sempre divide por espaço antes de aplicar o algoritmo
- **BPE (Geralmente):** o GPT-2 usa regras de pré-tokenização (ex.: nunca mescla letra com espaço)
- **SentencePiece (Opcional):** trata o espaço como caractere normal ( ) essencial para idiomas sem separação por espaço (japonês, chinês)



# Comparação de Métodos de Tokenization

|                   | Caractere | BPE         | WordPiece  | Unigram          |
|-------------------|-----------|-------------|------------|------------------|
| Direção           | —         | Bottom-up   | Bottom-up  | Top-down         |
| Critério de merge | —         | Frequência  | Likelihood | Baseado em perda |
| Tratamento de OOV | Total     | Chars raros | [UNK]      | Chars raros      |
| Determinístico    | Sim       | Sim         | Sim        | Não*             |
| Pre-tokenization? | Não       | Geralmente  | Sim        | Opcional         |
| Modelos notáveis  |           |             |            |                  |
|                   | ByT5      | Família GPT | BERT       | T5, LLaMA        |

\*O Unigram pode produzir múltiplas segmentações válidas; durante o treinamento, isso pode servir como forma de **subword regularization** (data augmentation).

## Qual Tokenizer Cada Modelo Usa?

---

| Modelo    | Tokenizer                 | Tamanho do Vocabulário |
|-----------|---------------------------|------------------------|
| BERT      | WordPiece                 | 30,522                 |
| GPT-2     | Byte-level BPE            | 50,257                 |
| GPT-3/4   | Byte-level BPE (tiktoken) | 100,256                |
| T5        | SentencePiece (Unigram)   | 32,000                 |
| LLaMA 1   | SentencePiece (BPE)       | 32,000                 |
| LLaMA 2/3 | SentencePiece (BPE)       | 32,000 / 128,256       |
| Gemma     | SentencePiece (BPE)       | 256,000                |
| Mistral   | SentencePiece (BPE)       | 32,000                 |
| Claude    | Byte-level BPE (custom)   | —                      |

**Tendência:** Modelos modernos estão migrando para vocabulários maiores e abordagens byte-level para melhorar a cobertura multilíngue.

# Subword Regularization e BPE Dropout

**Ideia:** A tokenization não precisa ser determinística durante o treinamento!

## Subword Regularization (Kudo, 2018)

O Unigram pode amostrar entre múltiplas segmentações válidas:

"lowest" pode ser:

- low est (probabilidade 0,7)
- lo we st (probabilidade 0,2)
- l o w est (probabilidade 0,1)

## BPE Dropout (Provilkov et al., 2020)

Ignora aleatoriamente alguns merges do BPE durante o treinamento:

Com dropout  $p = 0,1$ :

- Normal: low est
- Ignorado: lo w est
- Ignorado: low e st

Ambos atuam como **data augmentation**, tornando os modelos mais robustos a segmentações raras.

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
- 8. De Tokens a Embeddings**
9. Resumo

# De Token IDs a Embeddings

O tokenizer produz uma sequência de **inteiros** (IDs). O transformer precisa de **vetores densos**.

## A tabela de embedding

Uma matriz  $E \in \mathbb{R}^{|V| \times d}$ , onde cada linha é o vetor de um token:

$$\mathbf{e}_i = E[i] \in \mathbb{R}^d$$

`nn.Embedding` é uma **tabela de lookup**, não uma transformação linear — apenas seleciona uma linha.

| ID        | $\mathbf{e} \in \mathbb{R}^d$ |          |          |
|-----------|-------------------------------|----------|----------|
| 0         | 0.12                          | -0.43    | ...      |
| $\vdots$  | $\vdots$                      | $\vdots$ | $\vdots$ |
| 15496     | 0.31                          | -1.20    | ...      |
| $\vdots$  | $\vdots$                      | $\vdots$ | $\vdots$ |
| $ V  - 1$ | -0.07                         | 0.88     | ...      |

ID 15496  $\rightarrow$  linha 15496  $\rightarrow \mathbf{e} \in \mathbb{R}^{768}$

## Pipeline completo

texto  $\xrightarrow{\text{tokenizer}}$  [IDs]  $\xrightarrow{\text{nn.Embedding}}$  [vetores]  $\xrightarrow{+PE}$  entrada do transformer

# De Token IDs a Embeddings (código)

## GPT-2 com HuggingFace + PyTorch

```
from transformers import AutoTokenizer, AutoModelForCausalLM

tokenizer = AutoTokenizer.from_pretrained("gpt2")
model      = AutoModelForCausalLM.from_pretrained("gpt2")

ids = tokenizer("Olá mundo", return_tensors="pt").input_ids
# ids.shape -> (1, seq_len) -- tensor de inteiros

embs = model.transformer.wte(ids)    # nn.Embedding lookup
# embs.shape -> (1, seq_len, 768) -- tensor de floats
```

- `wte` = *word token embedding* — o nome usado internamente pelo GPT-2
- Após este passo, os embeddings posicionais são somados e a sequência entra na pilha de transformers
- O `model.transformer.wte` é uma instância de `torch.nn.Embedding(50257, 768)`

# Quantos Parâmetros Custam os Embeddings?

$$\text{params}_{\text{embedding}} = |V| \times d$$

| Modelo       | $ V $  | $d$   | Params emb. | Total | % do total  |
|--------------|--------|-------|-------------|-------|-------------|
| GPT-2 Small  | 50,257 | 768   | 38,6M       | 117M  | <b>33%</b>  |
| GPT-2 Medium | 50,257 | 1,024 | 51,5M       | 345M  | <b>15%</b>  |
| GPT-2 Large  | 50,257 | 1,280 | 64,3M       | 762M  | <b>8,4%</b> |
| GPT-2 XL     | 50,257 | 1,600 | 80,4M       | 1,5B  | <b>5,4%</b> |
| LLaMA 2 7B   | 32,000 | 4,096 | 131M        | 7B    | <b>1,9%</b> |

## Conclusões

- **Vocabulário maior = mais parâmetros:** escolhas de tokenization têm custo direto no tamanho do modelo
- **Modelos menores pagam mais:** em GPT-2 Small,  $\frac{1}{3}$  dos parâmetros estão na tabela de embedding
- **Weight tying:** GPT-2 e muitos outros modelos **reutilizam** a mesma matriz como camada de saída (`lm_head`) — sem custo extra de parâmetros para a projeção final

# Overview

---

1. Por que a Tokenization Importa
2. Abordagens Ingênuas
3. Byte Pair Encoding (BPE)
4. WordPiece
5. Unigram Language Model (SentencePiece)
6. Byte-Level BPE
7. Comparação e Considerações Práticas
8. De Tokens a Embeddings
- 9. Resumo**



# Principais Conclusões

---

1. **A tokenization é uma decisão arquitetural**, não uma etapa de pré-processamento
2. **Nível de caractere e de palavra** são extremos demais; métodos por subword encontram o ponto de equilíbrio
3. **BPE**: mescla os pares mais frequentes (bottom-up, baseado em frequência)
4. **WordPiece**: mescla pares maximizando a likelihood (bottom-up, baseado em probabilidade)
5. **Unigram**: começa grande, poda os tokens menos úteis (top-down, baseado em perda)
6. **SentencePiece**: um framework para tokenization agnóstica ao idioma (suporta BPE ou Unigram)
7. **Byte-level BPE**: garante cobertura total com 256 tokens base

# Referências

---

## Artigos principais:

- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *ACL*.
- Schuster, M. & Nakajima, K. (2012). Japanese and Korean Voice Search. *ICASSP*.
- Wu, Y. et al. (2016). Google's Neural Machine Translation System. *arXiv:1609.08144*.
- Kudo, T. (2018). Subword Regularization. *ACL*.
- Kudo, T. & Richardson, J. (2018). SentencePiece. *EMNLP*.

## Leitura complementar:

- Radford, A. et al. (2019). Language Models are Unsupervised Multitask Learners. (GPT-2)
- Xue, L. et al. (2022). ByT5: Towards a Token-Free Future. *NAACL*.
- Yu, L. et al. (2023). MegaByte: Predicting Million-Byte Sequences. *NeurIPS*.
- Provilkov, I. et al. (2020). BPE-Dropout. *ACL*.